



Instituto Politécnico Nacional
Escuela Superior de Computo



Base de Datos

Practica 1

Ejercicio 1

PARTE A

P R E S E N T A N:

EJERCICIO 1- CONTROL DE VERSIONES CON GIT

3CV2

A 18 de septiembre de 2026

EJERCICIO 1: CONTROL DE VERSIONES CON GIT Y GITHUB

Parte A: Investigación Conceptual y Fundamentos de Git

1. Qué es un sistema de control de versiones y qué problema resuelve en equipo

Un **Sistema de Control de Versiones (VCS)** es una infraestructura de software que actúa como una base de datos encargada de rastrear, gestionar y versionar los cambios efectuados sobre una colección de archivos digitales a lo largo del tiempo, almacenando múltiples registros de un solo archivo en momentos específicos. A diferencia de los sistemas tradicionales o de otros DVCS que almacenan diferencias de archivos a través del tiempo, Git opera mediante una secuencia de copias instantáneas (*snapshots*) de archivos en miniatura, con un diseño sencillo y eficiente para manejar proyectos masivos como el kernel de Linux.

Ya que sabemos que es un sistema de control de versiones, entremos un poco en detalle, antes la mayoría de VCS eran centralizados, esto daba lugar a problemas con el manejo de las versiones de un proyecto, ya que estaban en un servidor central al cuál te conectabas para poder descargar la versión actualizada de un proyecto, pero estos cambios eran individuales y podía haber conflictos con más de una persona trabajando en el proyecto, ahí nacen los **DVCS** o sistema de control de versiones distribuido por sus siglas en inglés, dicho sistema permitía tener una copia local de la base de datos en cada computadora, sincronizar los cambios mediante parches lo que permite trabajar en tiempo real, además permite trabajar sin conexión y comprometer los cambios para fusionarlos cuando se disponga de una red; entre otras muchas otras funciones que facilitan el trabajo colaborativo; más adelante, esto permitió la introducción de otros conceptos como rama, merge, pull request, entre otros relacionados.

Analizando otros problemas que soluciona un **DVCS** encontramos algunos como:

Colisiones y sobreescritura accidental:

Resuelve el riesgo crítico donde un ingeniero borra inadvertidamente las funciones desarrolladas por otro al intercambiar archivos comprimidos o compartidos en red, garantizando la integridad colaborativa del código.

Falta de trazabilidad histórica y auditoría:

Permite determinar con exactitud matemática qué autor modificó una línea de código específica, en qué fecha exacta, bajo qué contexto y motivado por qué necesidad técnica o corrección de error.

Capacidad de reversión (*rollback*):

Otorga la capacidad inmediata de revertir el sistema ante fallos en producción, restituyéndolo a una versión previamente validada y estable sin poner en riesgo la integridad global del proyecto [1].

Punto único de fallo de la centralización:

Los sistemas centralizados anteriores presentaban una debilidad crítica: una caída del servidor o corrupción del equipo implicaba la pérdida total e irreparable de los archivos. Los sistemas de control de versiones distribuidos (DVCS) —como Git, nacido para resolver la comercialización de BitKeeper— replican el repositorio íntegramente en cada cliente. Si el servidor central falla, cualquier clon local contiene todos los datos y puede restaurarlo por completo.

Desarrollo lineal restrictivo:

Permite y soporta eficientemente el desarrollo no lineal mediante miles de ramas paralelas simultáneas con alta velocidad, evitando que el trabajo concurrente de múltiples desarrolladores se bloquee entre sí.

2. Cuál es la diferencia entre Git y GitHub

Aunque suelen usarse erróneamente como sinónimos, **Git** y **GitHub** son entidades distintas que operan en capas diferentes del ciclo de desarrollo, a grandes rasgos, GitHub es el contenedor donde trabaja GIT guardando las versiones de un archivo, aquí se mencionan algunas de sus características:

Git (Software local de control de versiones):

Es un software libre de control de versiones distribuido (DVCS) creado en 2005 por Linus Torvalds y la comunidad de Linux tras perder el acceso gratuito a la herramienta que utilizaban para mantener el kernel mediante parches. Diseñado para ser rápido, eficiente con proyectos masivos y con soporte robusto para desarrollo no lineal (ramificación), Git opera localmente en la máquina del usuario sin requerir conexión a internet para registrar cambios o gestionar ramas. Estructuralmente, funciona mediante un grafo acíclico dirigido (DAG) que almacena instantáneas (*snapshots*) inmutables respaldadas por funciones hash criptográficas (SHA-1 y SHA-256).

GitHub (Plataforma colaborativa en la nube):

Es una plataforma de servicios web y computación en la nube fundada para alojar repositorios de código de cualquier desarrollador y adquirida en 2018 por Microsoft Corporation. Basada sobre la tecnología de Git, centraliza repositorios remotos y abarca todo el ciclo de vida del software (planificación, desarrollo, implementación y operación). Añade capacidades que Git no posee de forma nativa: gestión de incidencias (*Issue Tracking*), revisión colaborativa mediante *Pull Requests*, control granular de accesos,

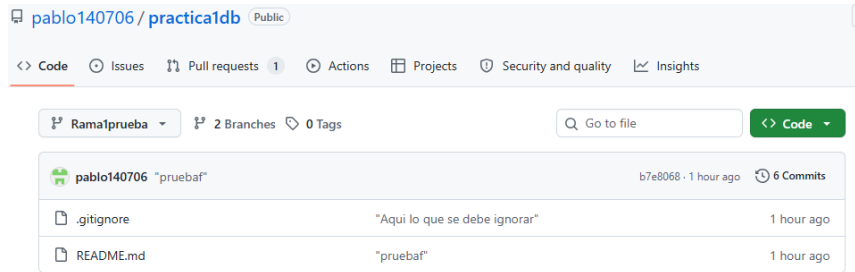
despliegue de sitios web (*GitHub Pages*) y automatización de integración/despliegue continuo (*GitHub Actions*).

Criterio	Git	GitHub
Naturaleza	Software de control de versiones distribuido (bajo nivel).	Plataforma web y servicio en la nube (gobernanza y colaboración).
Origen y Propiedad	Creado en 2005 por Linus Torvalds (código abierto / GPL).	Fundado para alojar código; adquirido en 2018 por Microsoft.
Entorno de ejecución	Local en la máquina del desarrollador.	En la nube (servidores remotos).
Dependencia de red	No requiere internet para registrar historial o gestionar ramas.	Requiere conexión a internet para sincronizar repositorios y colaborar.
Estructura interna	Grafo acíclico dirigido (DAG) con snapshots criptográficos (SHA-1/256).	Capa sobre Git que centraliza el árbol de ramas y datos remotos.
Funcionalidades clave	Commits, ramificación (branching), fusión (merging) y desarrollo no lineal.	Pull Requests, Issue Tracking, Actions (CI/CD) y GitHub Pages.

3. Definición de conceptos clave con ejemplos ilustrativos

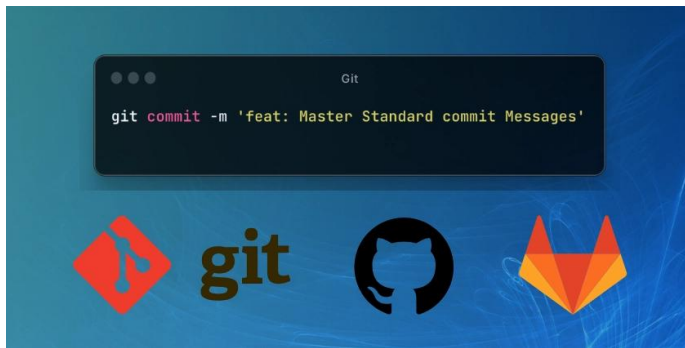
Repositorio (Repository): Es el contenedor o carpeta raíz donde vive todo tu proyecto junto con el historial completo de sus cambios, versiones y metadatos de Git (la carpeta oculta `.git`). Puede ser local (en tu máquina) o remoto (alojado en GitHub, GitLab, etc.).

Creas una carpeta llamada tienda-online, ejecutas git init y ahora esa carpeta rastrea cada modificación de tus archivos.



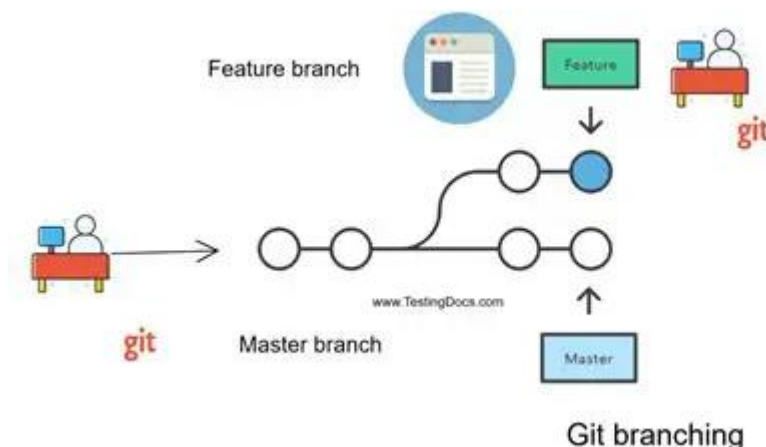
Confirmación (Commit): Es una instantánea (*snapshot*) del estado de tus archivos en un momento específico, acompañada de un identificador único (hash) y un mensaje descriptivo del cambio realizado. Funciona como un punto de restauración o un "guardar partida".

Modificas el formulario de registro y ejecutas `git commit -m "feat: validar longitud de contraseñas"`



Rama (Branch): Es una línea de desarrollo independiente que se bifurca del historial principal. Permite trabajar en nuevas funciones, corregir errores o experimentar sin alterar el código estable ni afectar el trabajo del resto del equipo.

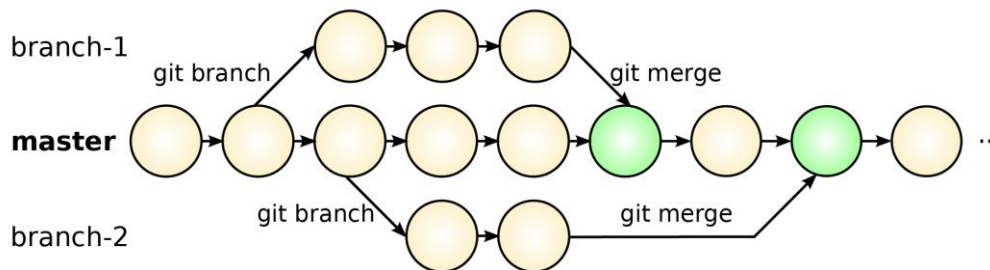
En lugar de modificar directamente la rama main, creas una rama llamada login-con-google para desarrollar esa funcionalidad aislada.



Fusión (Merge): Es la acción de integrar los cambios, commits y el historial de una rama dentro de otra (normalmente trayendo una rama de características a una rama base como main o develop).

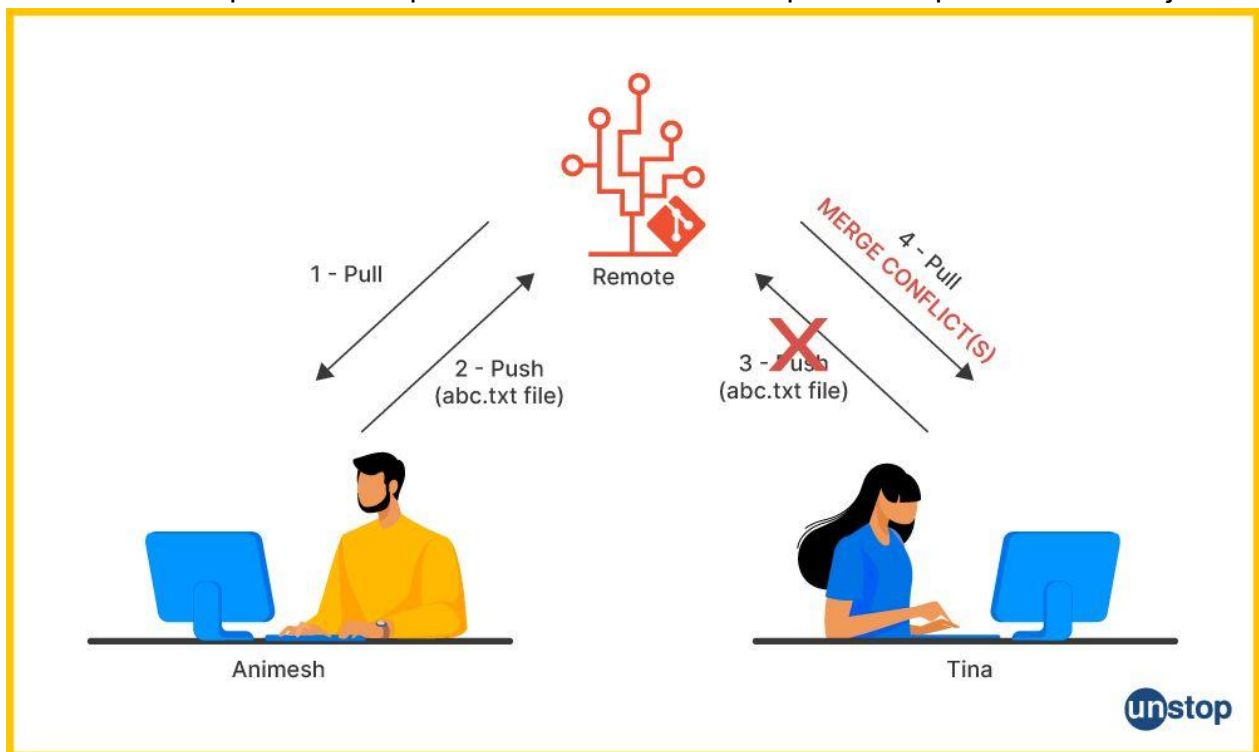
Terminaste la rama login-con-google, te sitúas en main y ejecutas `git merge login-con-google` para incorporar todo el trabajo a la versión principal.

Git



Conflicto de fusión (Merge Conflict): Ocurre cuando dos ramas modificaron exactamente las mismas líneas de un archivo o una rama eliminó un archivo que la otra editó, impidiendo que Git decida automáticamente cuál versión conservar. Requiere que un desarrollador elija manualmente el código final.

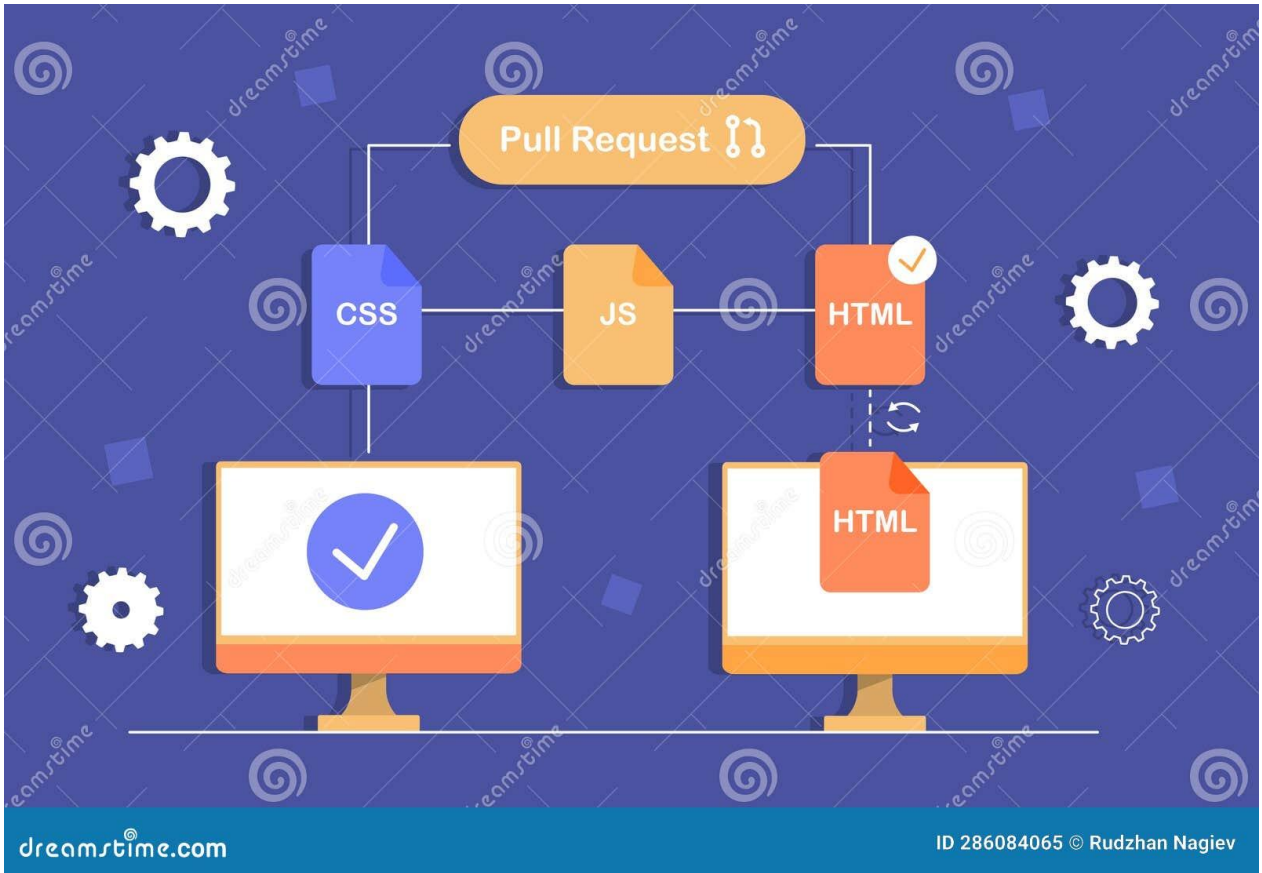
En la línea 12 de `app.js`, tú pusiste `const PORT = 3000`; en tu rama, pero un compañero puso `const PORT = 8080`; en main. Al hacer el merge, Git marca el conflicto para que decidan qué puerto dejar.



Pull Request (PR): Es una solicitud formal en plataformas remotas (como GitHub o GitLab) para fusionar una rama en otra. Sirve como espacio de revisión de código

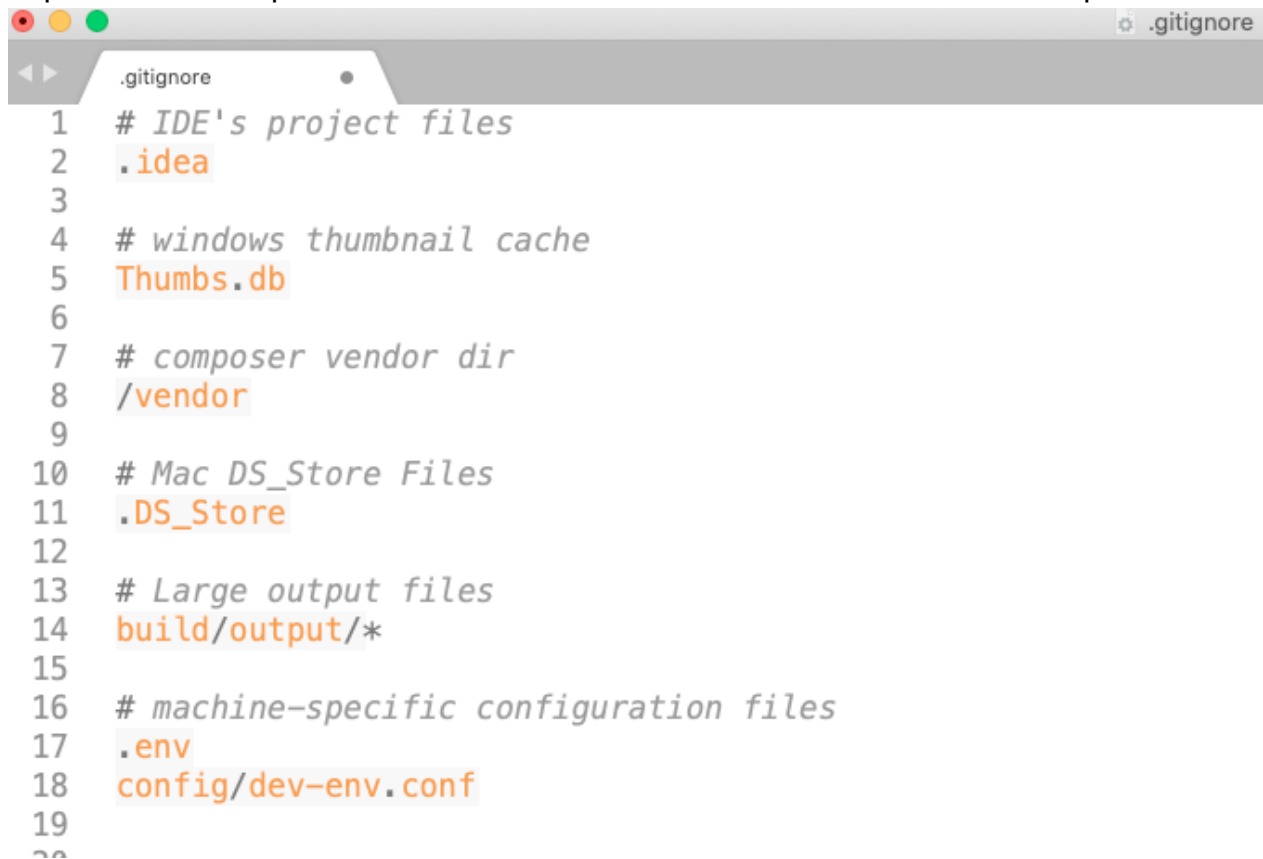
(code review), debate entre colaboradores y ejecución de pruebas automatizadas antes de autorizar la integración.

Subes tu rama login-con-google a GitHub y abres un Pull Request hacia main para que tu equipo revise que el código cumpla con los estándares antes de aprobarlo.



Archivo `.gitignore`: Es un archivo de texto plano ubicado en la raíz del proyecto que le indica a Git qué archivos, carpetas o extensiones debe ignorar deliberadamente para no incluirlos en el control de versiones.

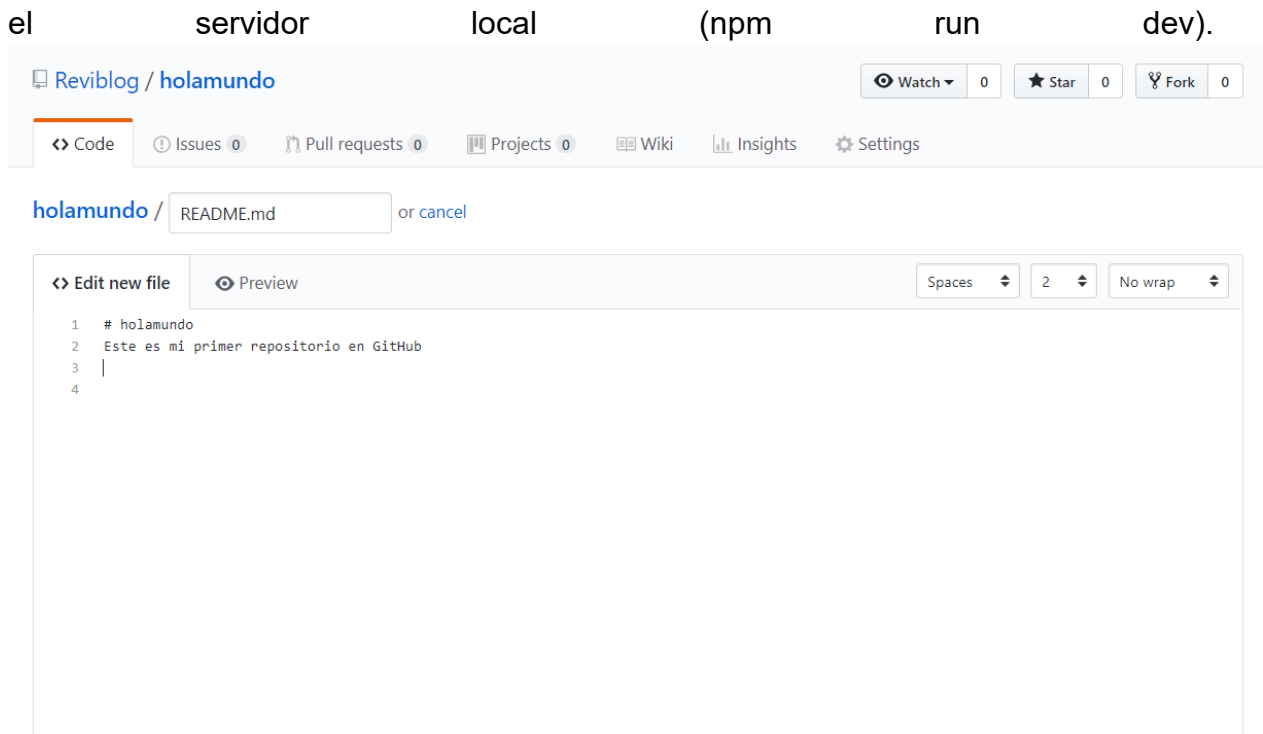
Incluir en el archivo las líneas `node_modules/`, `.env` o `build/` para evitar subir dependencias pesadas o credenciales secretas al repositorio.

A screenshot of a code editor window showing a .gitignore file. The window has a title bar with three colored buttons (red, yellow, green) on the left and a tab labeled '.gitignore' on the right. The code is as follows:

```
1  # IDE's project files
2  .idea
3
4  # windows thumbnail cache
5  Thumbs.db
6
7  # composer vendor dir
8  /vendor
9
10 # Mac DS_Store Files
11 .DS_Store
12
13 # Large output files
14 build/output/*
15
16 # machine-specific configuration files
17 .env
18 config/dev-env.conf
19
20
```

Archivo `README.md`: Es la carta de presentación y manual de usuario/desarrollador de un repositorio. Explica qué hace el proyecto, cómo instalarlo, cómo configurarlo, cómo ejecutarlo y qué tecnologías utiliza.

Un documento con secciones que explican: descripción del proyecto, requisitos previos (ej. Node.js 20), comando de instalación (`npm install`) y comando para iniciar



4. Flujo de trabajo basado en ramas y revisión entre pares (Peer Review)

Un flujo de trabajo moderno (*Branch-Based Workflow*, sustentado en GitHub Flow, GitLab Flow y adaptaciones de *Trunk-Based Development*) opera bajo una premisa inviolable: la rama troncal (main) constituye la única fuente de verdad (*single source of truth*), debe permanecer en estado **verde, desplegable y libre de regresiones**. Las confirmaciones directas sobre main se restringen mediante políticas de ramas protegidas (*branch protection rules*). Toda unidad de trabajo (corrección de defectos, migraciones de esquema o *features*) se confina a ramas de vida corta (*short-lived feature branches*), cuya integración se gestiona formalmente mediante un *Pull Request* (PR) o *Merge Request* (MR).

En el estado del arte de la ingeniería de software, la revisión por pares (*Peer Code Review*) actúa como una puerta de calidad (*quality gate*) no negociable por tres fundamentos respaldados empíricamente:

- **Detección temprana de anomalías y reducción del costo del cambio:** Conforme a la regla de Boehm y el enfoque *Shift-Left*, mitigar fallos funcionales, vulnerabilidades de seguridad o defectos de concurrencia durante la revisión cuesta hasta un orden de magnitud menos que subsanarlos en entornos productivos. En pipelines modernos, el revisor humano complementa el análisis

estático automatizado (linters, SAST) evaluando lógica de negocio y casos de borde complejos.

- **Mitigación del factor de autobús y soberanía colectiva:** El proceso democratiza el conocimiento arquitectónico y funcional del sistema. Elimina los silos de información (especialmente críticos en el diseño de persistencia, índices y modelos de datos), distribuyendo el contexto entre el equipo y garantizando la mantenibilidad a largo plazo sin dependencias unipersonales.
- **Consistencia técnica y observancia de estándares:** Fomenta la adherencia a convenciones de diseño, legibilidad, patrones de concurrencia y restricciones de integridad relacional (normalización, claves foráneas y transaccionalidad ACID). Asimismo, convierte cada PR en un mecanismo continuo de mentoría técnica y gobernanza del código.